

Crypto Tracker iOS App — Requirement Document

Objective

Build a production-style cryptocurrency tracking iOS application using modern iOS development practices.

The project is intended for:

- interview preparation
- GitHub portfolio showcase
- architecture demonstration
- modern SwiftUI + concurrency practice

Timeline target:

- 2 days

Primary focus:

- code quality
 - architecture
 - async handling
 - clean structure
 - interview discussion value
-

Tech Stack

UI

- SwiftUI

Architecture

- MVVM
- Repository Pattern
- Dependency Injection

Concurrency

- async/await
- MainActor

Networking

- URLSession
- Generic API Client

Persistence

- SwiftData OR UserDefaults (simple favorites persistence)

Logging

- OSLog or custom logger abstraction
-

App Features

1. Coin List Screen

Requirements

Display list of cryptocurrencies fetched from API.

UI Elements

- coin image
- coin name
- symbol
- current price
- 24h percentage change
- loading indicator
- error state
- retry button

Functional Requirements

- fetch initial page on launch
- support pull to refresh
- support pagination
- support search filtering
- support navigation to detail screen

Technical Requirements

- pagination threshold loading
- duplicate API call prevention
- async image loading

- state-driven UI
-

2. Coin Details Screen

Requirements

Display detailed information for selected cryptocurrency.

UI Elements

- coin image
- name
- current price
- market cap
- high/low values
- description
- optional mini chart
- favorite button

Functional Requirements

- fetch details from API
 - show loading state
 - show error state
 - persist favorite status
-

3. Favorites Screen

Requirements

Display locally saved favorite coins.

Functional Requirements

- add/remove favorites
- persist locally
- update UI immediately

Persistence Options

Preferred:

- SwiftData

Fallback:

- UserDefaults
-

API Requirements

API Provider

CoinGecko API

Base URL: <https://api.coingecko.com/api/v3/>

Architecture Requirements

Folder Structure

App/ Core/ Features/ Domain/ Resources/

Core Layer

Networking

Contains:

- APIClient
- Endpoint abstraction
- NetworkError
- request builder

Persistence

Contains:

- favorites persistence
- local storage logic

Logging

Contains:

- logger abstraction

- screen event logging
 - network failure logging
-

Domain Layer

Contains

- entities/models
- repository protocols

Example

CoinRepository protocol

Features Layer

Each feature contains:

- Views
- ViewModels
- Models (if feature-specific)

Example: Features/CoinList/ Features/CoinDetails/ Features/Favorites/

State Management

View State

All screens should use state-driven rendering.

Required state enum:

- idle
- loading
- success
- failure

Avoid:

- excessive boolean state flags
-

Networking Requirements

Generic Request Layer

Networking layer must support:

- generic decoding
- reusable endpoints
- proper error mapping

Required Error Cases

- invalid URL
 - decoding failure
 - no internet
 - server error
 - unknown error
-

Pagination Requirements

Behavior

Load next page when user scrolls near bottom.

Requirements

- prevent duplicate fetches
 - prevent infinite rapid requests
 - track current page
 - track loading state
-

Concurrency Requirements

Must Use

- `async/await`
- `@MainActor ViewModels`

Optional

- cancellation handling

- debounced search
-

Search Requirements

Behavior

Filter/search cryptocurrencies.

Requirements

- debounce optional
 - case insensitive search
 - update results dynamically
-

Logging Requirements

Log Events

- screen opened
 - API failure
 - retry action
 - pagination requests
-

UI Requirements

Must Support

- dark mode
- loading state
- empty state
- retry state

Design Goal

Minimal and clean.

Do not spend excessive time on animations or visual polish.

Non-Functional Requirements

Code Quality

- meaningful naming
- modular structure
- reusable components
- avoid massive ViewModels

Performance

- smooth scrolling
 - avoid duplicate rendering
 - avoid unnecessary API calls
-

Deliverables

Required

- working iOS application
 - GitHub repository
 - README.md
-

README Requirements

Include

- project overview
 - architecture explanation
 - tech stack
 - folder structure
 - screenshots
 - future improvements
-

Interview Discussion Topics

Developer should be able to explain:

- MVVM architecture

- repository pattern
 - dependency injection
 - async/await
 - MainActor
 - pagination handling
 - error handling
 - state-driven UI
 - persistence strategy
 - networking abstraction
 - tradeoffs made due to timeline
-

Scope Restrictions

Do NOT add:

- authentication
- websocket streaming
- notifications
- advanced animation systems
- overly generic architecture
- micro-modules
- excessive protocol abstraction

Primary goal: ship a polished, explainable, production-style sample app quickly.